

Study Group Platform Software Requirements Specification (SRS)

SE-2 Final Project

Team members:

- Rahma Shaban Elhusainy – 20230202
- Kareem Ayman Abdelgaber – 20230413
- Sama Mohamed Sayed – 20230264
- Bavly Aziz William – 20230119
- Mohamed Tarek Abdelaziz – 20230482
- Ismail Mahmoud Mansor - 20230083

Project Name	Study Group Platform
Architecture	Microservices Architecture
Frontend	React
Backend	Spring Boot
Messaging	Apache Kafka
Database	PostgreSQL
Security	JWT Authentication

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the functional and nonfunctional requirements of the **StudyGroup** platform. The document describes the system behavior, features, constraints, interfaces, and expected performance of the platform. It serves as a communication reference between stakeholders including developers, testers, project managers, instructors, and future maintainers.

The StudyGroup platform is designed to support collaborative learning among students by allowing them to create, join, and manage study groups. The system provides features such as authentication, group management, real-time discussions, educational material sharing, and administrative moderation. This SRS also acts as a foundation for system design, implementation, testing, and maintenance activities.

1.2 Document Conventions

The following abbreviations and conventions are used throughout this document:

Prefix	Meaning
FR	Functional Requirement
NFR	Nonfunctional Requirement
UC	Use Case
API	Application Programming Interface
JWT	JSON Web Token

1.3 Intended Audience

This document is intended for multiple stakeholders involved in the development and maintenance of the StudyGroup platform, including:

Developers:

Developers will use this document to understand system requirements, backend services, APIs, database structure, and frontend behavior during implementation.

Software Testers:

Testers will rely on the documented requirements to create test cases, validate functionalities, and ensure the platform behaves correctly under different conditions.

Project Managers:

Project managers will use this document to monitor project progress, allocate resources, estimate timelines, and ensure all requirements are fulfilled.

Instructors and Evaluators:

Instructors supervising the project can review the document to evaluate system completeness, architecture decisions, and implementation quality.

Future Maintainers:

Future developers and maintainers can use this SRS to understand the platform architecture and safely extend or modify the system.

1.4 Project Scope

StudyGroup is a web-based educational collaboration platform that enables students to participate in collaborative learning through online study groups.

The system allows:

- Students to browse and search study groups

- Group Creators to create and manage study groups
- Students to request joining groups
- Group members to exchange study materials
- Members to participate in real-time discussions and comments
- Admins to moderate users and groups

The platform supports both online and offline study sessions and includes information such as:

- Subject
- Meeting schedule
- Meeting type
- Group description
- Group materials
- Maximum number of members

The system uses a microservices architecture implemented using Spring Boot and Apache Kafka for scalable communication between services. Real-time communication is achieved using WebSocket technology.

The project aims to provide:

- Secure authentication and authorization
- Scalable backend services
- Real-time collaborative communication
- Easy educational resource sharing
- Efficient group management

1.5 References

The following references and technologies were used during the preparation and implementation of this project:

- IEEE Software Requirements Specification (SRS) Standards
- Spring Boot Official Documentation
- Apache Kafka Documentation
- PostgreSQL Documentation
- React Documentation
- Docker Documentation
- WebSocket API Documentation
- JWT Authentication Standards
- REST API Design Guidelines

2. Overall Description

2.1 Product Perspective

StudyGroup is a distributed web application built using a microservices architecture. The system consists of multiple independent backend services implemented using Spring Boot.

Communication between services is handled asynchronously through Apache Kafka messaging queues.

The frontend is implemented using React to provide a responsive and interactive user experience. PostgreSQL databases are used for persistent data storage, while Docker containers simplify deployment and scalability.

The platform integrates:

- Authentication Service

- Group Management Service
- Discussion Service
- Material Sharing Service
- Admin Moderation Service

The system supports real-time communication using WebSocket connections for discussions and live interactions between members.

2.2 Product Features

The StudyGroup platform provides the following major features:

User Authentication

- User registration and login
- JWT-based authentication
- Role-based authorization

Study Group Management

- Create, update, delete, and manage study groups
- Group approval workflow handled by admins
- Maximum member limit support

Group Discovery

- Browse available study groups
- Search groups by:
 - Subject
 - Location

- Meeting time

Membership Management

- Students can request to join groups
- Group Creators can approve or reject requests

Real-Time Discussions

- Real-time messaging using WebSockets
- Group discussions and comments

Material Sharing

- Upload and download study files
- Share educational resources within groups

Administrative Features

- Approve or reject Group Creator accounts
- Moderate and approve study groups
- Monitor system operations

2.3 User Classes and Characteristics

Student:

Students are the main users of the platform. They can:

- Browse study groups without authentication
- Register and log in
- Request joining groups
- Participate in discussions

- Access shared materials

Students are expected to have basic knowledge of web applications.

Group Creator:

Group Creators are responsible for managing study groups. They can:

- Create study groups
- Edit group details
- Accept or reject join requests
- Upload study materials
- Moderate group discussions

Group Creators require approved accounts from the Admin before accessing creator features.

Admin:

Admins supervise and moderate the platform. They can:

- Approve or reject creator accounts
- Approve or reject study groups
- Monitor reports and activities
- Manage system integrity

Admins have full system access and advanced privileges.

2.4 Operating Environment

The platform operates in a web-based distributed environment with the following technologies:

Frontend:

- React.js
- HTML5
- CSS3
- JavaScript

Backend:

- Spring Boot Microservices
- REST APIs
- WebSocket Communication

Database:

- PostgreSQL

Infrastructure:

- Docker Containers
- Apache Kafka Message Broker

Supported Platforms:

- Windows
- Linux
- macOS
- Modern web browsers

2.5 Design and Implementation Constraints

The following constraints must be followed during implementation:

- Spring Boot must be used for backend services
- PostgreSQL must be used as the primary database
- Kafka must handle service communication
- JWT authentication is mandatory
- Docker must be used for deployment
- Real-time communication must use WebSockets
- RESTful API standards must be followed

The system should maintain modularity to support future scalability and maintenance.

2.6 Assumptions and Dependencies

Assumptions:

- Users have stable internet access
- Users use modern browsers supporting WebSockets
- Servers remain operational during usage

Dependencies:

- Apache Kafka services must be running correctly
- PostgreSQL database servers must be available
- Docker environment must be properly configured
- External libraries and frameworks must remain compatible

3. Functional Requirements

Use case diagram:

{ }

Detailed Use-Cases Description :

FR-1 User Registration

The system shall allow users to create accounts using email, username, and password.

Description

Users must provide valid information during registration. Passwords shall be encrypted before storage.

Actors

- Student
- Group Creator

Preconditions

- User is not authenticated

Postconditions

- Account is created successfully
 - JWT token may be generated after login
-

FR-2 User Login

The system shall authenticate users using JWT authentication.

Description

Users shall log in using email and password credentials. Upon successful authentication, a JWT token shall be issued.

Actors

- Student
- Group Creator
- Admin

Preconditions

- User account exists

Postconditions

- User session becomes active
-

FR-3 Create Study Group

The system shall allow Group Creators to create and manage study groups.

Group Information Includes

- Creator Name
- Subject
- Description
- Maximum Members
- Meeting Type
- Meeting Schedule
- Group Materials

Actors

- Group Creator
-

FR-4 Join Study Group

Students shall send requests to join study groups.

Description

A student can request membership in a study group after authentication.

Actors

- Student
-

FR-5 Review Join Requests

Group Creators shall approve or reject join requests.

Description

The creator receives pending requests and decides whether to accept or reject them.

Actors

- Group Creator
-

FR-6 Real-Time Discussion

Users shall exchange real-time messages using WebSocket communication.

Description

Members inside the same study group can participate in live discussions and comments.

Actors

- Student
 - Group Creator
-

FR-7 Material Sharing

Users shall upload and access educational materials.

Description

Group members can share files such as PDFs, presentations, and notes.

Actors

- Student
- Group Creator

4. External Interface Requirements

4.1 User Interfaces

The platform provides responsive web interfaces including:

- Registration Page
- Login Page
- Study Group Listing Page
- Search Interface
- Group Dashboard
- Discussion Chat Interface
- File Sharing Interface
- Admin Dashboard

The UI shall support desktop and mobile screen sizes.

4.2 Hardware Interfaces

The system requires:

- Internet-enabled client devices
- Servers capable of running Docker containers
- Sufficient storage for PostgreSQL databases and uploaded files

4.3 Software Interfaces

The platform integrates with:

- PostgreSQL Databases
- Apache Kafka
- REST APIs
- WebSocket Protocols
- JWT Authentication Services

4.4 Communication Interfaces

The system uses:

- HTTP/HTTPS protocols
- RESTful APIs
- Kafka messaging queues
- WebSocket real-time communication

5. Nonfunctional Requirements

5.1 Performance Requirements

- API response time shall not exceed 2 seconds under normal load.
- The system shall support at least 1000 concurrent users.
- Real-time discussions shall maintain low latency communication.

5.2 Safety Requirements

- The system shall prevent invalid data corruption.
- Backup and recovery mechanisms shall be available.
- Failures shall not result in permanent data loss.

5.3 Security Requirements

- Passwords shall be encrypted before storage.
- JWT authentication shall secure API access.
- Role-based authorization shall restrict unauthorized actions.
- HTTPS communication shall be enforced.

5.4 Software Quality Attributes

Maintainability

The codebase shall follow modular and clean architecture principles.

Scalability

Microservices and Kafka shall support horizontal scaling.

Reliability

The platform shall maintain stable operation with minimal downtime.

Availability

The system shall remain accessible 24/7 except during maintenance.

Usability

Interfaces shall be user-friendly and responsive.

6. System Evolution

6.1 Anticipated Changes

Future versions may include:

- AI-based study group recommendations
- Push notifications
- Analytics dashboards
- Mobile applications
- Video meeting integration
- Attendance tracking

6.2 Future Design Impact

The microservices architecture enables:

- Easy addition of new services
- Independent service deployment
- Horizontal scaling
- Improved maintainability
- Better fault isolation

Kafka and Docker support flexible future expansion.

7. Requirements Discovery and Validation

7.1 Discovery Approaches

Requirements were collected using:

- Interviews with students
- Brainstorming sessions
- Competitor platform analysis
- UI prototyping
- Educational collaboration research

7.2 Validation Techniques

The following validation methods were used:

- Requirement reviews
- Prototype validation
- Functional testing
- Traceability analysis
- Stakeholder feedback sessions

Conclusion

The StudyGroup platform provides a scalable, secure, and collaborative learning environment for students and educators. By integrating real-time communication, study material sharing, and group management features within a distributed microservices architecture, the platform enhances collaborative education while maintaining scalability, reliability, and maintainability for future growth.